

Jeez.Fitness 系统架构演进

概述

本文档记录了 Jeez.Fitness 系统从技术选型到架构设计的演进过程，以及核心设计决策的背景和理由。

技术架构选型

前后端分离架构

项目采用前后端完全分离的架构模式：

- 后端开发：由本人负责
- 前端开发：由何俊杰负责

后端技术选择：Java vs .Net Core

技术对比分析

技术栈	优势	劣势
Java	生态成熟，开发者众多	-
.Net Core	微软背书，代码可靠性强	学习成本高，使用者较少

最终决策

选择：**Java**

综合考虑团队规模、技术积累和长期维护成本，最终选定 Java 作为后端开发语言。

框架架构决策

问题分析

Java 生态中存在两条技术路线：

- 使用成熟开源框架（如若依、Jeecg-Boot）：高度封装、功能完善，但存在过度设计问题
- 从零开始搭建框架：需要更多人力投入，但更加轻量灵活

决策因素

- 当前系统规模：仅单服务器部署，暂不涉及分布式场景
- 框架需求：不需要服务发现、服务注册、配置中心等复杂组件
- 开发效率：借助 Claude Code 进行智能辅助开发，提升整体效率

最终选择

自建轻量化框架

核心优势：

- 减少不必要的功能封装和依赖，系统更加轻量
- 团队能够充分掌握架构细节，便于长期维护
- 更加专注于架构设计和业务实现
- 借助 AI 辅助开发工具，提高开发效率

核心模块设计

Auth 认证授权模块

Auth 模块是整个系统的核心基础模块，负责用户身份认证和权限管理。

权限控制方案：RBAC

RBAC (Role-Based Access Control)：基于角色的访问控制

核心设计理念：

- 将系统权限和业务操作以角色为单位进行组织
- 每个用户可以拥有多个角色
- 用户的操作权限由其所拥有的角色决定

核心优势：

- 权限管理灵活，易于扩展
- 用户权限配置简洁：只需为用户分配相应角色
- 角色复用率高，维护成本低

技术实现

采用 **Sa-Token + JWT + Redis** 三层结合实现：

- **Sa-Token**：提供完整的权限认证框架和 RBAC 支持
- **JWT**：实现无状态身份验证，Token 内部包含用户信息
- **Redis**：缓存 Token 和权限数据，支持：
 - 主动撤销 Token (用户登出立即失效)
 - 实时权限更新 (权限变更无需等待 Token 过期)
 - 并发登录控制 (防止重复登录、强制登出等)
 - 分布式部署下的会话共享

为什么需要 Redis：Token 有效期长达 30 天，如果仅依赖 JWT，用户注销后 Token 仍然有效，无法真正登出。通过 Redis 维护 Token 黑名单和权限缓存，确保权限变更的实时生效。

Redis 扩展性设计

当前采用单机 Redis 部署，但系统架构已为未来扩展预留空间。

阶段一（当前）：单机 Redis

- 适用于用户规模 < 10 万
- 部署简单，足以满足现有业务需求
- 成本最低

阶段二：Redis Sentinel（高可用）

- 触发条件：Redis CPU 使用率 > 80% 或内存接近上限
- 特性：自动故障转移，主从复制
- 适用规模：10-100 万用户
- 优势：增强可靠性，无需水平扩展

阶段三：Redis Cluster（水平扩展）

- 触发条件：用户规模 > 100 万或需要突破性能瓶颈
- 特性：自动数据分片（16384 个 slots），多节点并发
- 适用规模：百万级以上用户
- 优势：无限扩展容量，高吞吐量

优化策略：

- 本地缓存：使用 Caffeine 本地缓存权限数据，减少 Redis 访问
- 活动超时：设置合理的 Token 活动超时时间，自动清理不活跃数据
- Pipeline 优化：批量操作时使用 Redis Pipeline，减少网络往返
- Lua 脚本：复杂操作使用 Lua 脚本保证原子性

数据存储架构

数据库选型：MySQL vs PostgreSQL

为什么选择 MySQL 而不是 PostgreSQL

虽然 PostgreSQL 最近很火，但 MySQL 仍是最佳选择，原因如下：

1. 系统特点决定选择

你的健身系统是典型的关系型数据库应用：

- 数据结构简洁明确：用户、教练、课程、订单
- 都是标准的行列式关系数据
- 不需要 JSONB、数组、复杂嵌套结构
- 不需要地理空间（PostGIS）、全文搜索等高级功能

这些都是 MySQL 能完美支持的场景。

2. 性能完全足够

指标	MySQL	PostgreSQL	说明
简单查询	☆☆☆☆☆	☆☆☆☆☆	MySQL 更快 5-10%

指标	MySQL	PostgreSQL	说明
写入吞吐	☆☆☆☆☆	☆☆☆☆	MySQL 更快 15-20%
单机极限	10-50K QPS	5-30K QPS	MySQL 更高

你的需求：10-100 万用户，5,000-10,000 QPS MySQL 单机可以支持，甚至有冗余。

3. 生态和运维成本

方面	MySQL	PostgreSQL
国内使用	99% 互联网公司	少数公司
分布式方案	ShardingSphere、MyCat (成熟完善)	Citus (付费)、Shardman (试验阶段)
高可用方案	MHA、Orchestrator (成熟稳定)	Patroni、repmgr (选择少)
人才储备	易找，费用低	稀缺，薪资高 20-30%
运维工具	丰富 (Percona、PT Toolkit)	有限
云服务支持	RDS、PolarDB 完善	选择较少

4. 成本考虑

数据量 100GB 场景：

MySQL：

- 服务器配置：4 核 8GB 内存
- 云 RDS 成本：¥500/月

PostgreSQL：

- 服务器配置：4 核 16GB 内存 (内存占用大)
- 云数据库成本：¥800/月

差异：每月多花 ¥300，仅为支持不需要的高级功能

5. PostgreSQL 适用场景

如果你的系统有以下需求，才应该选 PostgreSQL：

- ☒ 需要存储大量 JSON 文档 (用户定制数据)
- ☒ 需要数组、范围类型
- ☒ 需要复杂分析查询 (窗口函数、CTE)
- ☒ 需要地理空间支持 (PostGIS)
- ☒ 需要强大的全文搜索

你的系统：都不需要。

扩展路径：MySQL 足以应对所有规模

用户规模	数据库方案	原因
< 10 万	单机 MySQL	性能过剩
10-100 万	主从读写分离	方案成熟，扩展清晰
100-1000 万	分库分表 + 缓存	ShardingSphere 完善
> 1000 万	云原生 (PolarDB)	无需管理底层

所有扩展方案都基于 MySQL，无需后悔选择。

MySQL 数据库设计

当前采用单机 MySQL 部署，存储所有业务数据。

单机承载能力评估

基于系统业务特性分析：

业务数据	增长规律	3年预期量	数据大小	状态
会员数据	年增 3K-1W 人	5-50 万人	25-250 MB	✅ 安全
教练数据	相对稳定	50-500 人	< 10 MB	✅ 安全
订单数据	人均 20 单/年	100-300 万单	300-900 MB	✅ 安全
访问日志	日增 100 万条	10 亿条	~200 GB	⚠️ 需分表

结论：单机 MySQL 可以支撑 **10-100 万** 用户规模。

容量扩展计划

阶段一（当前）：单机 MySQL

- 适用规模：< 10 万用户
- 关键指标：CPU < 60%，连接数 < 300，QPS < 1000
- 准备工作：定期备份、监控慢查询、规划数据增长

阶段二：MySQL 主从读写分离

- 触发条件：
 - CPU 使用率持续 > 70%
 - 慢查询数量 > 100/小时
 - 并发连接数 > 500
- 适用规模：10-100 万用户
- 部署：1 个主库 + 2 个从库
- 优势：分散读取压力，提高并发能力
- 成本：增加 2 台服务器

阶段三：分库分表 + 中间件

- 触发条件：

- 单表数据量 > 5000 万
- 并发写入 > 5000/秒
- 总数据量 > 100 GB
- 适用规模：100-1000 万用户
- 方案：按用户 ID 分片 (hash) · 按时间分表 (日志)
- 中间件：ShardingSphere、DBLE 等
- 优势：解决写入瓶颈 · 支持水平扩展
- 成本：4-8 台服务器 + 中间件

阶段四：云数据库或微服务

- 适用规模：> 1000 万用户
- 方案：迁移至云数据库 (如 PolarDB) 或拆分微服务
- 优势：完全托管 · 自动扩展
- 成本：根据用量按需付费

性能优化策略

索引优化：

- 为高频查询字段添加索引 (member_id、coach_id、create_time)
- 定期分析慢查询并优化

日志表管理：

- 按时间分表 (按月或按季度)
- 定期归档和清理历史日志

连接池配置：

- 使用 HikariCP · 最大连接数 = (core_count × 2) + effective_spindle_count
- 定期监控连接使用情况

缓存策略：

- 热数据使用 Redis 缓存 (权限、用户信息)
- 减少数据库查询频率

MySQL 性能瓶颈诊断和优化

当 MySQL 出现性能瓶颈时，需要系统的诊断和优化方案。

性能瓶颈识别

常见症状：

- 慢查询 (单个查询 > 1s)
- 连接数爆满 (Too many connections)
- 磁盘 I/O 打满 (iowait > 50%)
- 内存占用过高 (> 服务器内存 50%)
- 锁竞争导致吞吐量下降

诊断命令：

```
-- 查看慢查询
SHOW PROCESSLIST; -- 实时进程
SELECT * FROM mysql.slow_log; -- 慢查询日志

-- 查看性能指标
SHOW STATUS LIKE 'Questions'; -- 总查询数
SHOW GLOBAL STATUS LIKE 'Threads_connected'; -- 连接数
SHOW STATUS LIKE 'Innodb_buffer_pool%'; -- 缓冲池使用

-- 分析执行计划
EXPLAIN SELECT * FROM bookings WHERE user_id = 100;
EXPLAIN FORMAT=JSON SELECT * FROM bookings WHERE user_id = 100;
```

5 大常见瓶颈和解决方案

瓶颈 1：慢查询（最常见）

*问题：*某些查询耗时过长，拖累整体性能

诊断：

```
-- 查看执行计划
EXPLAIN SELECT * FROM bookings WHERE user_id = 100 AND course_id = 50;
-- 关注 type（应该是 ref 或 range，不应该是 ALL）
```

解决方案：

- 添加复合索引：`CREATE INDEX idx_user_course ON bookings(user_id, course_id);`
- 优化 SQL：避免函数处理（如 `DATE(created_at)` 应改为 `created_at BETWEEN ... AND ...`）
- 使用覆盖索引：索引包含所有 SELECT 字段，无需回表

*效果：*通常可以提升 10-100 倍

瓶颈 2：连接数过多

*问题：*新连接建立失败，出现 "Too many connections" 错误

诊断：

```
-- 查看当前连接数
SELECT COUNT(*) FROM information_schema.PROCESSLIST;

-- 查看上限
SHOW VARIABLES LIKE 'max_connections';
```

解决方案：

- 增加 `max_connections` : `SET GLOBAL max_connections = 1000;`
- 优化连接池：设置 HikariCP 的 `maximum-pool-size = (核心数 × 2) + 1`
- 杀死长时间占用的连接：`KILL <process_id>;`

效果：立即解决连接溢出

瓶颈 3：磁盘 I/O 过高

问题：磁盘 I/O 成为瓶颈，查询异常慢

诊断：

```
# 查看磁盘 I/O
iostat -x 1 5
# 关注 await (> 10ms 表示磁盘繁忙)
```

解决方案：

- 最优：升级到 **SSD** (效果最好，一次投入)
- 索引优化：确保查询使用了索引，减少随机 I/O
- 增大 `innodb_buffer_pool_size`：减少磁盘访问

效果：SSD 可提升 100-500 倍，索引优化可提升 10-100 倍

瓶颈 4：内存占用过高

问题：MySQL 进程占用内存过多，导致 OOM

诊断：

```
-- 查看 Buffer Pool 命中率
SHOW STATUS LIKE 'Innodb_buffer_pool%';
-- 命中率 = (read_requests - read_ahead) / read_requests (应 > 99%)
```

解决方案：

- 合理设置 `innodb_buffer_pool_size = 总内存 × 75%`
- 不要设置过大 (留 25% 给 OS 和其他进程)
- 考虑使用外部缓存 (Redis) 替代 MySQL 的内部缓存

效果：正确配置可减少 30-50% 的磁盘 I/O

瓶颈 5：锁竞争

问题：并发事务互相等待，吞吐量无法提高

诊断：


```
-- 查看锁等待
SELECT * FROM information_schema.INNODB_LOCK_WAITS;

-- 查看长事务
SELECT * FROM information_schema.INNODB_TRX
WHERE TRX_REQUESTED_LOCK_ID IS NOT NULL;
```

解决方案：

- 避免大事务：分批处理，每批 1000-10000 行
- 使用行锁而非表锁：确保使用 InnoDB 引擎
- 缩短事务时间：提前计算，事务内只做数据库操作
- 合理的索引：减少 gap lock 和 next-key lock

效果：可提升 2-10 倍

优化优先级

按照这个优先级优化，效果最好：

优先级	优化项	效果
● 高	添加缺失索引	10-100x
● 高	升级到 SSD	100-500x
● 中	优化 SQL 语句	10-50x
● 中	增加 Buffer Pool	2-10x
○ 低	调整参数	1-5x
○ 低	使用缓存层	5-20x (整体系统)

长期优化方案

第一阶段 (当前)：单机 MySQL 优化

- 索引设计和优化
- SQL 语句优化
- 参数配置优化
- 使用 SSD 硬盘

第二阶段 (10 万用户)：主从读写分离

- 搭建主从复制
- 读流量路由到从库
- 主库专处理写
- 添加缓存层 (Redis)

第三阶段 (100 万用户)：分库分表

- 选择分片键 (如 user_id)
- 设计分片方案 (4-8 个库)
- 使用 ShardingSphere 中间件
- 跨库查询优化

监控告警：

- 慢查询监控：记录所有 > 1s 的查询
- 连接数监控：告警阈值设为 max_connections 的 80%
- 磁盘空间监控：剩余空间 < 20% 时告警
- 内存使用监控：MySQL 内存占用 > 服务器总内存 60% 时告警

缓存架构设计

Redis + MySQL 结合防止业务量大的方法

当业务量增大时，单纯的 MySQL 会成为瓶颈。通过合理的缓存架构，可以有效地应对高并发。

三层缓存模型



各层职责：

- **L1 本地缓存 (Caffeine)**：进程内存，最快，容量小 (100MB-1GB)
- **L2 分布式缓存 (Redis)**：进程外，快速，容量中等 (1GB-10GB)
- **L3 数据库 (MySQL)**：持久化存储，最慢，容量无限

缓存策略对比

策略	适用场景	优点	缺点
旁路缓存	用户、订单等关键数据	灵活，易于更新	可能有短暂不一致
写穿缓存	必须强一致的数据	强一致性	写入慢
异步写回	日志、统计数据	写入快	可能丢数据

具体应用场景

场景一：用户和教练信息 (旁路缓存)

- 读取时：查本地 → 查 Redis → 查 MySQL → 写回缓存
- 更新时：更新 MySQL → 删除缓存
- 缓存时间：24 小时

场景二：权限和角色数据 (旁路缓存 + 定时刷新)

- 用户登录时加载到 Redis
- 权限变更立即清除缓存
- 缓存时间：30 天（由 Token 驱动）

场景三：订单统计数据（延迟删除）

- 先写 MySQL，后异步更新统计缓存
- 缓存失效时间：5 分钟
- 允许短暂数据不一致

场景四：访问日志（纯 Redis 异步）

- 实时写入 Redis 列表
- 后台定期批量写入 MySQL（每 5 分钟或 1000 条）
- 提高日志写入吞吐量

缓存雪崩和穿透防护

缓存穿透（恶意查询不存在数据）：

- 方案：缓存空值或使用布隆过滤器
- 原理：将不存在的 Key 也缓存起来（设置较短过期时间）

缓存击穿（热点数据过期）：

- 方案：使用互斥锁或延期缓存
- 原理：热点数据过期时，只让一个线程重新加载，其他线程等待

缓存雪崩（大量缓存同时过期）：

- 方案：随机过期时间或异步刷新热点数据
- 原理：避免大量缓存同时失效

监控指标

指标	目标值	作用
Redis 命中率	> 95%	评估缓存有效性
本地缓存命中率	> 80%	评估本地缓存有效性
MySQL QPS	< 5000	确保数据库不过载
Redis 内存使用	< 80%	防止内存溢出

实施建议

第一阶段（当前）：

- 部署 Redis 单机
- 为用户、教练等关键数据实现三层缓存
- 权限数据必须走缓存，减少数据库压力

- 配置监控和告警

第二阶段（用户 > 10 万）：

- 访问日志改为异步批处理
- 统计数据改为延迟更新
- 实现布隆过滤器防缓存穿透
- 优化缓存过期策略

第三阶段（用户 > 100 万）：

- 升级 Redis Sentinel 高可用
- 实现热点数据识别和预加载
- 使用本地缓存减少 Redis 访问
- 考虑使用 Redis Cluster

消息队列架构

订单系统与 MQ 的结合

当系统需要处理高并发订单时，使用消息队列实现异步解耦是必需的。

技术选型：Kafka vs RocketMQ

特性	Kafka	RocketMQ
吞吐量	☆☆☆☆☆ 百万级	☆☆☆☆ 百万级
消息延迟	毫秒级	微秒级（更低）
消息顺序性	⚠️ 局部有序	✅ 支持全局有序
事务性消息	✅ 0.11+ 支持	✅ ☆ 原生强支持
延迟队列	✖️ 无原生支持	✅ 原生支持 18 档
消息追踪	⚠️ 需插件	✅ 内置完整支持
国内最佳实践	中等	非常广泛（阿里出品）

结论：选择 RocketMQ

原因：

1. 订单必须有序：同一订单的消息要严格按时间顺序处理（创建 → 支付 → 发货）
2. 订单需要事务：支付成功与消息投递必须原子性保证
3. 订单需要延迟：支持自动取消未支付订单（30 分钟），RocketMQ 原生支持
4. 需要消息追踪：用于问题定位和审计

核心问题与解决方案

问题 1：消息顺序性

同一订单的消息必须有序处理（创建 → 支付 → 发货 → 确认）。

解决：

- 按订单 ID 分组，相同订单路由到同一队列
- 使用顺序消费模式，单线程处理每个队列

问题 2：消息可靠性（事务一致性）

支付成功后必须投递消息，否则订单更新失败。

解决：RocketMQ 事务型消息

- 生产者发送“半消息”（消费者看不到）
- 执行本地事务（数据库扣费）
- 确认/回滚消息，保证原子性

问题 3：消费幂等性

消费者宕机重启时可能重复消费消息，导致数据重复处理。

解决：

- 消息去重表：记录已处理的消息 ID
- 消费前检查，已处理直接跳过
- 使用数据库唯一索引防止重复插入

问题 4：消费延迟和堆积

消费速度跟不上生产速度，消息不断堆积。

解决：

- 优化消费逻辑，批量处理
- 动态扩容消费者实例
- 监控消费延迟，及时告警

问题 5：死信队列处理

消息重试多次仍然失败（如外部服务宕机），需要人工处理。

解决：

- 设置最大重试次数（如 3 次）
- 超过阈值发送到死信队列
- 死信队列人工审查处理

问题 6：顺序性与并发的平衡

顺序消费吞吐量低，但需要保证订单有序。

解决：分治法

- VIP 订单独立队列（更多消费线程）

- 普通订单普通队列
- 延迟订单延迟队列

部署和监控

推荐部署：

- RocketMQ Broker：3-5 个节点（高可用）
- NameServer：3 个节点（注册中心）
- 消费者：按需扩容（可多个实例）

关键监控指标：

- 消费延迟：不超过 1 分钟
- 消息堆积：< 10,000 条
- 死信队列：需要告警
- 消费异常率：< 0.1%

分布式锁设计

分布式锁的应用场景

在高并发系统中，分布式锁用于防止数据不一致和超卖等问题。

系统中的具体场景

场景 1：课程预约（库存扣减）

- 问题：多用户同时报名，可能超卖
- 例如：10 个座位，11 个用户抢购
- 解决：锁定课程 ID，严格控制预约人数

场景 2：优惠券秒杀

- 问题：优惠券可能发放超数
- 例如：100 张优惠券，可能被领出 102 张
- 解决：锁定优惠券 Code，确保精确发放

场景 3：会员卡充值

- 问题：并发充值导致次数计算错误
- 例如：应该加 20 次，只加了 10 次
- 解决：锁定会员卡，保证充值原子性

场景 4：教练提现

- 问题：余额不足仍然可能被提现多次
- 例如：1000 元，同时提现两笔 600 元
- 解决：锁定账户，检查余额后原子扣减

场景 5：用户信息更新

- 问题：并发修改导致信息丢失
- 例如：同时改昵称和头像，可能其中一个被覆盖
- 解决：细粒度锁，分别锁定不同字段

场景 6：积分兑换

- 问题：兑换失败时积分和收益不同步
- 例如：积分扣了，课程没加
- 解决：锁定积分账户，原子执行

实现方案对比

方案	实现	可靠性	性能	复杂度	推荐
MySQL 行锁	FOR UPDATE	☆☆☆☆☆	☆☆	中	☆☆☆
Redis 单机	SETNX	☆☆☆	☆☆☆☆☆	低	☆☆☆☆
Redisson	基于 Redis	☆☆☆☆☆	☆☆☆☆☆	中	☆☆☆☆☆
Zookeeper	临时顺序节点	☆☆☆☆☆	☆☆	高	☆☆

最优选择：Redisson

- 自动续期（Watch Dog 机制），不会意外释放
- 支持可重入，防止死锁
- 支持公平锁和读写锁，优化性能
- 与 Redis 无缝集成，依赖现有基础设施

优化策略

1. Watch Dog 自动续期

- 业务执行中，Redisson 自动检测并延期锁
- 无需手动计算业务耗时
- 保证只要业务未完成，锁就不会过期

2. 可重入锁防死锁

- 同一线程可多次获取同一锁
- 获取 n 次，释放 n 次
- 避免自己与自己的死锁

3. 细粒度锁优化并发

- 不锁整个对象，而锁具体字段
- 例如：不锁 user:123，而锁 user:123:name、user:123:avatar
- 不同字段的修改可以并发进行

4. 读写锁区分场景

- 多个读操作并发执行

- 写操作独占
- 适合读多写少的场景（如权限数据）

5. 缓存 + 锁组合

- 热数据走缓存（无锁，毫秒级）
- 冷数据写时获取分布式锁
- 99% 的读请求无需锁，性能极优

6. 本地锁 vs 分布式锁

- 单机操作：使用 synchronized（本地锁）
- 分布式操作：使用 Redisson（分布式锁）
- 降低分布式锁的使用频率和复杂度

实施建议

第一阶段（当前）：

- 核心场景（库存、金额）：使用 Redisson 分布式锁
- 普通场景：使用 synchronized 本地锁
- 监控锁竞争情况

第二阶段（用户增长）：

- 优化热点数据：缓存 + 锁结合
- 改进：细粒度锁，减少锁竞争
- 监控：锁等待时间、获取失败率

第三阶段（超高并发）：

- 考虑乐观锁（版本号）替代悲观锁
- 使用读写锁优化读多写少场景
- 分析链路，尽量减少临界区

Spring Cloud Alibaba 微服务架构设计

概述

当系统规模增长到百万级用户时，单体架构面临瓶颈。微服务架构通过服务拆分、独立部署、按需伸缩来应对高并发和高可用需求。

Spring Cloud Alibaba 是国内最成熟的微服务框架套件，由阿里巴巴开源，包含 6 个核心组件：

组件	功能	适用场景
Nacos	服务发现、配置管理	服务注册与负载均衡
Sentinel	限流、熔断、降级	保护系统稳定性
Dubbo	高性能 RPC 框架	内部服务通信

组件	功能	适用场景
Seata	分布式事务	跨库跨服务数据一致性
RocketMQ	消息队列	异步解耦、峰值削平
SkyWalking	链路追踪、监控	可观测性和性能分析

Spring Cloud Alibaba vs Spring Cloud 对比

特性	Spring Cloud Alibaba	Spring Cloud
服务发现	Nacos (功能多)	Eureka (功能少)
配置中心	Nacos (热更新)	Spring Cloud Config (需 Bus)
限流熔断	Sentinel (功能完整)	Hystrix (已停用)
RPC 框架	Dubbo (高性能)	OpenFeign (HTTP 基础)
分布式事务	Seata (国产)	无原生方案
消息队列	RocketMQ (高可靠)	无 (需整合第三方)
国内支持	完善 (阿里出品)	一般 (社区驱动)

结论：对于国内互联网公司，**Spring Cloud Alibaba 是更好的选择**，尤其是涉及分布式事务、消息队列、链路追踪等高级功能时。

